



字符串



什么是字符串？



字符串 是由**多个字符**组成的序列。

- 在 C 语言中，字符串用字符数组 `char[]` 表示
- 在 C++ 中，更推荐使用 `string` 类型（需要包含 `<string>` 头文件）

对比	C 风格字符串	C++ <code>string</code>
类型	<code>char[]</code> 或 <code>char*</code>	<code>std::string</code>
长度获取	<code>strlen()</code> 函数	<code>.length()</code> 方法
拼接	<code>strcat()</code> 函数	<code>+</code> 运算符
内存管理	手动	自动

💡 **生活类比：**字符串就像一串**珍珠项链**，每一颗珍珠就是一个字符，整条项链就是一个字符串。

引入 string 类型



使用 *string* 需要包含头文件：

```
#include <string>    // string 类型的头文件
#include <iostream>  // 输入输出
using namespace std;
```

注意： *string* 属于 C++ 标准库，不是 C 语言的内容。

💡 很多 OJ 题解中会使用 `#include <bits/stdc++.h>`，它包含了所有标准库，但实际开发中建议按需引入。

string 的声明与初始化



```
string s1;           // 声明一个空字符串
string s2 = "hello"; // 声明并初始化为 "hello"
string s3("world");  // 另一种初始化方式
string s4 = s2;       // 拷贝构造, s4 = "hello"
```

常用构造函数:

写法	说明
<code>string s;</code>	空字符串
<code>string s = "abc";</code>	用字符串字面量初始化
<code>string s(n, 'c');</code>	用 n 个字符 c 初始化

```
string s(5, 'A'); // s = "AAAAA"
```



字符串的输入



方式一：*cin* >> (默认行为)

```
string s;
```

```
cin >> s; // 读取一个单词，遇到空格/换行停止
```

输入：hello world → s = "hello"

方式二：*getline()* (读取整行)

```
string s;
```

```
getline(cin, s); // 读取一整行，包括空格
```

输入：hello world → s = "hello world"

⚠ **注意：** `getline()` 会读取换行符之前的所有内容，**不会**把换行符存入字符串。



字符串的输出



使用 *cout* 直接输出:

```
string s = "Hello, C++!";  
cout << s; // 输出: Hello, C++!  
cout << s << endl; // 输出并换行
```

综合示例

```
#include <iostream>  
#include <string>  
using namespace std;  
  
int main() {  
    string name;  
    cout << "Input your name: ";  
    getline(cin, name);  
    cout << "Hello, " << name << "! " << endl;  
    return 0;  
}
```

字符串的长度 —— *size()* 和 *length()*



string 提供了两个方法来获取长度：

```
string s = "hello";
```

```
cout << s.length(); // 输出 5
```

```
cout << s.size(); // 输出 5 (效果相同)
```

方法	说明
<i>s.length()</i>	返回字符串长度，习惯用于字符串
<i>s.size()</i>	返回字符串长度，STL 通用风格

💡 **注意：** *size()* 和 *length()* 的返回值类型是 `size_t`（无符号整数），**不是 `int`**。

```
int len = s.size(); // 可以这样用，但可能会有警告
```



访问字符串中的字符 —— [] 运算符



字符串中的每个字符可以通过下标访问，下标从 0 开始：

```
string s = "hello";  
cout << s[0];    // 输出 'h'  
cout << s[1];    // 输出 'e'  
cout << s[4];    // 输出 'o'
```

遍历字符串：

```
for (int i = 0; i < s.size(); i++) {  
    cout << s[i] << endl;    // 逐个输出每个字符  
}
```

⚠ 注意：最后一个字符的下标是 $s.size() - 1$ ，不是 $s.size()$ 。

字符串的拼接 —— + 运算符



string 支持用 + 直接拼接：

```
string s1 = "Hello";  
string s2 = "World";  
string s3 = s1 + " " + s2;    // s3 = "Hello World"
```

拼接时要注意类型：

```
string s = "abc";  
s = s + 'd';           // ✓ 正确: s = "abcd"  
s = s + 1;             // ✗ 错误: 不能直接拼接整数  
s = s + to_string(1);  // ✓ 正确: s = "abcd1"
```

💡 to_string() 可以将数字转换为字符串。

字符串常用操作速览



操作	方法	示例
获取长度	<code>s.size() / s.length()</code>	<code>s.size() → 5</code>
访问字符	<code>s[i]</code>	<code>s[0] → 'h'</code>
拼接	<code>s1 + s2</code>	<code>"Hello" + "World"</code>
判空	<code>s.empty()</code>	空返回 <code>true</code>
反转	<code>reverse(s.begin(), s.end())</code>	需要 <code>< algorithm ></code>
查找子串	<code>s.find("sub")</code>	返回位置或 <code>-1</code>
追加字符	<code>s.push_back('c')</code>	尾部添加字符



实战①： P5705 数字反转



题目描述

输入一个浮点数（如 123.4），将其数字反转后输出（如 4.321）。

输入格式：一个浮点数（如 123.4）

输出格式：反转后的数字（如 4.321）

思路

1. 将数字**当作字符串**读入
2. 将字符串**反转**
3. 直接输出反转后的字符串

这种方法比用数学方法处理浮点数简单得多！



P5705 方法一：使用 reverse() 函数



```
#include <iostream>
#include <string>
#include <algorithm>    // std::reverse
using namespace std;

int main() {
    string s;
    cin >> s;           // 读入 "123.4"
    reverse(s.begin(), s.end()); // 反转: s = "4.321"
    cout << s << endl;  // 输出 4.321
    return 0;
}
```

`reverse()` 是 C++ STL 中的通用反转函数，对 `string` 同样有效。

P5705 方法二：手动倒序输出



```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s;
    cin >> s;
    // 从最后一个字符开始，倒着输出
    for (int i = s.size() - 1; i >= 0; i--) {
        cout << s[i];
    }
    cout << endl;
    return 0;
}
```

💡 **注意：**循环条件 $i \geq 0$ ，由于 i 是 `int` 类型，当 i 变为 -1 时循环结束。
如果写成 `for (int i = s.size(); i >= 0; i--)` 会出错（因为 `s.size()` 是无符号数）。

实战②：P1914 凯撒密码



题目描述

小书童忘记了密码，但记得密码是由原文字符串中**每个字母向后移动 n 位**形成的。 z 的下一个字母是 a ，如此循环。

输入格式：

- 第一行：整数 n （移动位数）
- 第二行：原文字符串（仅含小写字母，长度 ≤ 50 ）

输出格式：加密后的密码

知识点

- 字符与 ASCII 码的算术运算
- 循环移位： $z \rightarrow a$ 的处理
- 取模运算处理 $n > 26$ 的情况



ASCII 码回顾



字符	ASCII 值
'a'	97
'b'	98
...	...
'z'	122

规律：

- 小写字母的 ASCII 码是**连续的** (97 ~ 122)
- 向后移动 n 位 $\rightarrow$ ASCII 码 + n
- 超过 'z' (122) 时，需要**回到** 'a' (97)

P1914 算法



对每个字符 c :

1. 计算移动后的 ASCII 码: $c + n$
2. 如果超过 'z' (122), 减去 26 回到 'a'

```
char newChar = c + n;  
if (newChar > 'z') {  
    newChar = newChar - 26;  
}
```

但是! 如果 $n > 26$ 怎么办?

移动 27 位 和 移动 1 位效果相同 → 先取模: $n = n \% 26$

P1914 参考代码



```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int n;
    string s;
    cin >> n >> s;
    n = n % 26;    // 只需要移动 n%26 位

    for (int i = 0; i < s.size(); i++) {
        if (s[i] + n > 'z') {
            s[i] = s[i] + n - 26;    // 超过 z 则回到 a
        } else {
            s[i] = s[i] + n;
        }
    }
    cout << s << endl;
    return 0;
}
```

P1914 代码逐行解析



```
int n;  
string s;  
cin >> n >> s;           // 读取 n 和字符串  
n = n % 26;                // 避免 n 大于 26 导致多次循环  
  
for (int i = 0; i < s.size(); i++) {  
    // 判断: 当前字符向后移动 n 位后是否超过 'z'  
    if (s[i] + n > 'z') {  
        s[i] = s[i] + n - 26;    // 超过则减去 26 回到 'a'  
    } else {  
        s[i] = s[i] + n;        // 未超过直接加  
    }  
}
```

输入	n	处理	输出
1 + abc	1	a→b, b→c, c→d	bcd
27 + abc	27%26=1	同上	bcd
1 + zzz	1	z→a	aaa

实战③：P1125 笨小猴



题目描述

输入一个单词，统计其中**出现次数最多的字母**和**出现次数最少的字母**的出现次数。

如果 $\text{maxn} - \text{minn}$ 是质数，输出 *LuckyWord* 和这个差值；否则输出 No Answer 和 0。

输入格式：一个单词（仅含小写字母，长度 < 100 ）

输出格式：两行

知识点

- 字符串遍历与字符统计
- 求最大值和最小值
- 质数判断



P1125 示例分析



样例 1

输入: *error*

字母	出现次数
e	1
r	3
o	1

$\text{maxn} = 3, \text{minn} = 1, \text{maxn} - \text{minn} = 2 \rightarrow 2$ 是质数 $\rightarrow$ 输出 *LuckyWord* 和 2



P1125 示例分析



样例 2

输入: *olympic*

字母	出现次数
所有字母	各 1 次

$\text{maxn} = 1, \text{minn} = 1, \text{maxn} - \text{minn} = 0 \rightarrow 0$ 不是质数 $\rightarrow$ 输出 No Answer 和 0

P1125 思路



方法：使用数组统计

由于只有 26 个小写字母，可以用一个长度为 26 的数组来统计每个字母的出现次数。

```
int cnt[26] = {0};    // 初始化全为 0

for (int i = 0; i < s.size(); i++) {
    cnt[s[i] - 'a']++;    // 统计对应字母的次数
}
```

$s[i] - 'a'$ 将字符 $'a' \sim 'z'$ 映射到下标 $0 \sim 25$

P1125 参考代码 (上)



```
#include <iostream>
#include <string>
using namespace std;

// 判断是否为质数
bool isPrime(int x) {
    if (x <= 1) return false;
    for (int i = 2; i < x; i++) {
        if (x % i == 0) return false;
    }
    return true;
}

int main() {
    string s;
    cin >> s;
    int cnt[26] = {0};    // 统计 26 个字母的出现次数

    for (int i = 0; i < s.size(); i++) {
        cnt[s[i] - 'a']++;
    }
```

P1125 参考代码 (下)



```
int maxn = 0, minn = 100;    // 初始化

for (int i = 0; i < 26; i++) {
    if (cnt[i] > 0) {        // 只考虑出现过的字母
        if (cnt[i] > maxn) maxn = cnt[i];
        if (cnt[i] < minn) minn = cnt[i];
    }
}

int diff = maxn - minn;
if (isPrime(diff)) {
    cout << "Lucky Word" << endl << diff << endl;
} else {
    cout << "No Answer" << endl << 0 << endl;
}
return 0;
}
```

字符串 vs 字符数组



特性	string	char[]
头文件	<code>< string ></code>	无需额外头文件
长度获取	<code>.size() / .length()</code>	<code>strlen()</code>
赋值	<code>s = "hello"</code>	<code>strcpy()</code>
拼接	<code>s1 + s2</code>	<code>strcat()</code>
比较	<code>s1 == s2</code>	<code>strcmp()</code>
越界访问	可能抛出异常	未定义行为
推荐	✅ C++ 推荐	⚠️ C 风格

💡 在 C++ 编程和 OJ 刷题中，**优先使用** `string`，更安全、更方便。



字符串常见陷阱



1. *cin* 不能读取空格

```
string s;  
cin >> s; // 输入 "hello world" → s = "hello"
```

✓ 解决: 使用 *getline(cin, s)*

2. *getline* 与 *cin* 混用的问题

```
int n;  
string s;  
cin >> n;  
getline(cin, s); // 会读取残留的换行符!
```

✓ 解决: 在 *cin >> n* 后加 *cin.ignore()* 或使用两次 *getline*

3. 下标越界

```
string s = "abc";  
cout << s[3]; // 越界! 合法下标是 0,1,2
```

✓ 解决: 始终确保 $i < s.size()$

谢谢大家！



每一个字符，都是信息世界的基本粒子

Q & A